

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INTERACTIVE TREE SIMULATOR

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	Assessment:	Assessment Tool 1 – Interactive Tree Simulator
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	<i>Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.</i>		
Student Name:	RAGUL.M	Reg. No.:	192524066
Section / Batch:	SLOT A	Date:	29/07/2026

Code of Conduct

I RAGUL.M (192524066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: RAGUL.M

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic		Focus	Q Nos	Marks
Binary Search Tree		Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals		Traversal types, In order, Post order and Pre order	2	20
AVL Tree		Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree		Properties, structures and Operations	4	20
Red Black Tree		Properties, Example and Operations	5	20
GRAND TOTAL:			100 Marks	

Annexure A – Interactive Tree Simulator

- Construct a Binary Search Tree (BST) by inserting the following keys: 50, 30, 70, 20, 40, 60, 80
Perform:
 - Inorder Traversal
 - Search for 60
 - Delete 30
 - Display the final BST.
- Complete Tree Traversals by Inserting 50, 30, 70, 20, 40, 60, 80. Find all the traversals.
 - Inorder
 - Preorder
 - Postorder
- Insert 70, 50, 90, 40, 60, 80, 100, 55 into an AVL Tree. Delete 40 and 90. Analyze how the tree rebalances after each deletion.
- Construct a B-Tree of order 3 by inserting 45, 25, 65, 15, 35, 55, 75
- Insert 100, 80, 120, 60, 90, 110, 130 into a Red-Black Tree. Analyze their heights.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations

Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, node split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.

[← Back to Home](#)

Binary Search Tree

Input

Input an array to convert to a BST.

50, 30, 70, 20, 40, 60, 80

Enter numbers separated by commas, e.g.: 5, 3, 10, 1, 6, 14

[▶ Visualize](#)

[Balance the Tree](#)

[Generate Python Code](#)

```
graph TD; 50((50)) --> 30((30)); 50 --> 70((70)); 30 --> 20((20)); 30 --> 40((40)); 70 --> 60((60)); 70 --> 80((80));
```

➤ AFTER BALANCING

[← Back to Home](#)

Binary Search Tree

Input

Input an array to convert to a BST.

50, 30, 70, 20, 40, 60, 80

Enter numbers separated by commas, e.g.: 5, 3, 10, 1, 6, 14

[▶ Visualize](#)

[Balance the Tree](#)

[Generate Python Code](#)

```
graph TD; 50((50)) --> 30((30)); 50 --> 70((70)); 30 --> 20((20)); 30 --> 40((40)); 70 --> 60((60)); 70 --> 80((80));
```

© 2025 Saifulah Bin Yusuf. All rights reserved.

BST has been balanced!

2. PRE ORDER

[← Back to Home](#)

Tree Traversals

50302040706080

50

30

70

20

40

60

80

Input

Input is level-by-level order.

50, 30, 70, 20, 40, 60, 80

Enter numbers separated by commas. Use null for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

IN ORDER

[← Back to Home](#)

Tree Traversals

20304050607080

50

30

70

20

40

60

80

Input

Input is level-by-level order.

50, 30, 70, 20, 40, 60, 80

Enter numbers separated by commas. Use null for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

POST ORDER

[← Back to Home](#)

Tree Traversals

20403060807050

50

30

70

20

40

60

80

Input

Input is level-by-level order.

50, 30, 70, 20, 40, 60, 80

Enter numbers separated by commas. Use null for empty nodes.

Traversal Order

Pre

In

Post

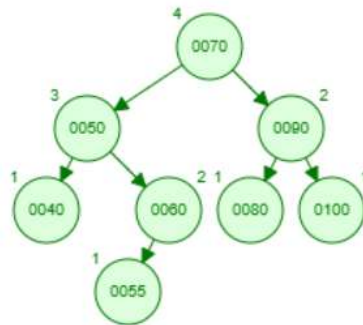
Generate Default

Visualize

Start Simulation

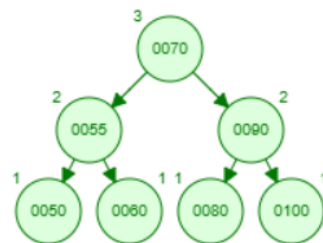
3.AVL TREE

AVL Tree



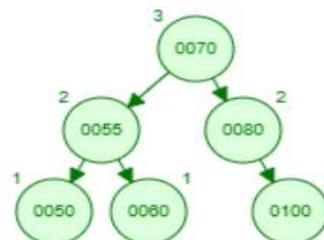
AFTER DELETING 40

AVL Tree



AFTER DELETING 90

AVL Tree

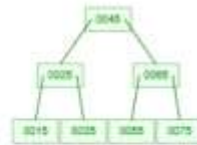


4.

B-Trees

☒ Max. Degree = 3 Preemptive Split / Merge (Even max degree only)

☐ Max. Degree = 4
☐ Max. Degree = 5
☐ Max. Degree = 6
☐ Max. Degree = 7



5.

Red/Black Tree

☐ Show Null Leaves

